4
Linear Regression

The previous chapter covered classification with categorical (discrete) features. We will now jump to the opposite supervised learning problem: regression with real-valued features. The technique in this chapter, **linear least squares**, is ubiquitous across quantitative disciplines. The exposition in this chapter is from a machine learning, discriminative model point of view. We follow the hypothesis space and loss function approach mentioned in Chapter 2 which we will also follow for subsequent chapters.

Problem Formulation

Our goal is to produce a regressor that predicts y from $\vec{x}$. This function, $f(\vec{x})$, could take any form. But for this chapter we will assume it is a linear function. That is, we fix the hypothesis space to be the set of all linear functions. We will find ways of relaxing this assumption in later chapters.

A linear function of multiple inputs (the elements of $\vec{x}$) takes the form

$$f(\vec{x}) = f(x_1, x_2, \ldots, x_n) = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + w_0$$

where w_0 is the "intercept" term and the others are the slopes along each axis. We can rewrite this as

$$\begin{aligned} &= w_0 + \sum_{i=1}^{n} w_i x_i && \text{swapping } w_0 \text{ to the front} \\ &= w_0 + \vec{w}^\top \vec{x} && \text{noting the sum is an inner product} \end{aligned} \tag{4.1}$$

where $\vec{w}$ is a vector of the slope values all stacked together.

The coefficients $w_0, w_1, w_2, \ldots, w_n$ are the parameters that define the function f. It is these values that we need to learn from the training data. In machine learning, such parameters are frequently called **weights**. $f(\vec{x})$ is obviously a function of $\vec{x}$. But, it is also a function

Key point
The parameters or coefficients of f are called weights.

of the weights; that is just not apparent from the notation. During learning we consider changing the weights. However, once learning has finished, the weights are fixed and only $\vec{x}$ varies (as we use the function to predict the output for different inputs).

The fact that w_0 is treated differently than the other w_i values in Equation 4.1 is slightly annoying. All of the math that follows works perfectly well with the form above. However, it is more convenient to fit w_0 into the vector with the other weights:

$$\begin{aligned} f(\vec{x}) &= w_0 + \vec{w}^\top \vec{x} \\ &= w_0 + \begin{bmatrix} w_1 & w_2 & \dots & w_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} \\ &= \begin{bmatrix} w_0 & w_1 & w_2 & \dots & w_n \end{bmatrix} \begin{bmatrix} 1 \\ x_1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} \\ &= \vec{w}^\top \vec{x} = \vec{x}^\top \vec{w} \end{aligned} \tag{4.2}$$

Sometimes $\vec{w}^\top \vec{x}$ is more convenient. Sometimes $\vec{x}^\top \vec{w}$ is.

where we have *redefined* $\vec{w}$ to include as its first (zeroth?) element the intercept term w_0, and we have *redefined* $\vec{x}$ to include an initial (zeroth?) term that is a constant value of 1. From here on out, we will assume that the vector $\vec{x}$ has already been so modified (a 1 placed in the beginning) and we will just try to estimate a function of the form $\vec{w}^\top \vec{x}$.

Key point
We add an initial constant feature of 1 to correspond to the intercept parameter.

Each choice of $\vec{w}$ corresponds to a different linear function of $\vec{x}$. So which $\vec{w}$ should we pick? We will select the weights that result in the smallest error (or loss) on the training data. To measure loss, we use the square of the difference between the predicted value and the true value. This makes the per-item loss function

$$l(y, f) = (y - f)^2 \tag{4.3}$$

where y is the true value (target output) and f is the predicted value (the result of $f(\vec{x})$).

The total **empirical loss** is the sum of the per-item loss over every example in the training data:

$$\begin{aligned} L &= \sum_{i=1}^{m} l(y_i, f(\vec{x}_i)) \\ &= \sum_{i=1}^{m} (y_i - f(\vec{x}_i))^2 \end{aligned} \tag{4.4}$$

Loss Minimization

Equation 4.4 defines the loss (error) of any chosen linear function. Our goal is to find the linear function that minimizes this loss. Each

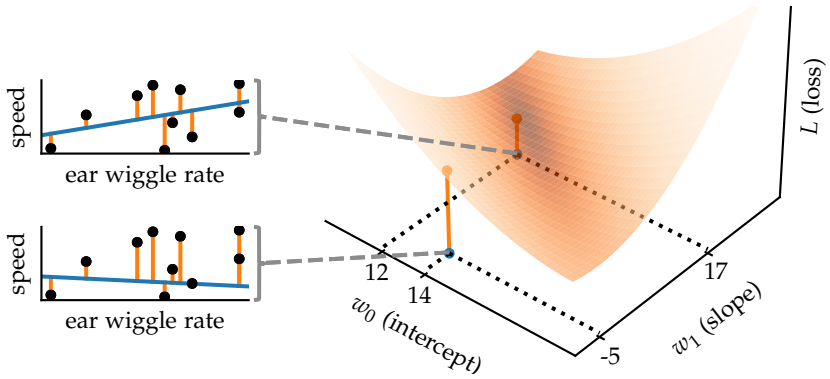

Figure 4.1: (right) Example of loss function for linear least squares, using only the second feature (ear wiggle rate) of the data in Table 2.1. (left) Two sample lines corresponding to the two marked points on the right.

such function is parameterized by our weights, $\vec{w}$, so this amounts to finding the vector $\vec{w}$ that, when plugged into f results in L being as small as possible.

Example Visualization

Figure 4.1 is a visualization of this for a simple one-dimensional regression problem. The goal is to predict the speed of a hippopotamus from its ear wiggle rate (the data from Table 2.1). On the right is a plot of the loss as a function of the two weights: $\vec{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$. Each point on this plot is a different $\vec{w}$ and therefore a different potential line. The height of the surface above the point is the total squared loss for that particular choice of line.

On the left of the figure are two plots corresponding to two $\vec{w}$s showing the corresponding lines (blue) and the distances from the line to each of the data points (red). The sum of the squares of these red distances is the height of the corresponding point on the right side of the figure.

Our goal is to find the weights that minimize the loss function. From the plot on the right, we can see this is very near to the upper plotted line with $\vec{w} = \begin{bmatrix} 12 \\ 17 \end{bmatrix}$.

If we had more features, $\vec{w}$ would have more dimensions and the plot on the right side of this figure would be in higher dimensions (and therefore not easily visualized).

Loss Derivatives and the Gradient

To find the weight vector $\vec{w}$ that minimizes Equation 4.4 we will take the (partial) deriviatives of the loss with respect to each of the

weights and set them all simultaneously to zero. This means that, locally, no change to any of the components of $\vec{w}$ will change the loss. In general, such a point could be a minimum, maximum, or saddle point. In this case, it is guarenteed to be a minimum. See Exercise 5.5.

We first need to write the loss in terms of $\vec{w}$:

$$L = \sum_{i=1}^{m} (y_i - f(\vec{x}_i))^2 = \sum_{i=1}^{m} (y_i - \vec{x}_i{}^\intercal \vec{w})^2 \tag{4.5}$$

Consider taking the partial derivative of this with respect to the zeroth weight, w_0:

$$\begin{aligned}
\frac{\partial L}{\partial w_0} &= \frac{\partial \sum_{i=1}^{m} (y_i - \vec{x}_i{}^\intercal \vec{w})^2}{\partial w_0} \\
&= \sum_{i=1}^{m} \frac{\partial (y_i - \vec{x}_i{}^\intercal \vec{w})^2}{\partial w_0} && \text{the derivative of a sum is the sum of the derivatives} \\
&= \sum_{i=1}^{m} 2 (y_i - \vec{x}_i{}^\intercal \vec{w}) \frac{\partial (y_i - \vec{x}_i{}^\intercal \vec{w})}{\partial w_0} && \text{chain rule} \\
&= \sum_{i=1}^{m} 2 (y_i - \vec{x}_i{}^\intercal \vec{w}) (-x_{i,0}) \ . && (4.6)
\end{aligned}$$

To see why this last step is true, note that $\vec{x}_i{}^\intercal \vec{w} = x_{i,0} w_0 + x_{i,1} w_1 + x_{i,2} w_2 + \cdots + x_{i,n} w_n$. Therefore, its derivative with respect to w_0 is $x_{i,0}$ because only the first term, $x_{i,0} w_0$, has w_0 in it. Thus, at our solution we want that

Further information
In general, $\frac{\partial \vec{a}^\intercal \vec{b}}{\partial b_k} = a_k$

$$0 = \frac{\partial L}{\partial w_0} = -2 \sum_{i=1}^{m} x_{i,0} (y_i - \vec{x}_i{}^\intercal \vec{w}) \ . \tag{4.7}$$

The same pattern holds for the other partial derivatives with respect to the other weights $(w_1, w_2, \ldots, w_n)$. Rearranging the terms slightly, we then get the following system of equations, setting the partial derivatives for each weight equal to zero.

$$\begin{aligned}
0 = \frac{\partial L}{\partial w_0} &= -2 \sum_{i=1}^{m} \boxed{x_{i,0}} (y_i - \vec{x}_i{}^\intercal \vec{w}) \\
0 = \frac{\partial L}{\partial w_1} &= -2 \sum_{i=1}^{m} \boxed{x_{i,1}} (y_i - \vec{x}_i{}^\intercal \vec{w}) \\
&\vdots \\
0 = \frac{\partial L}{\partial w_n} &= -2 \sum_{i=1}^{m} \boxed{x_{i,n}} (y_i - \vec{x}_i{}^\intercal \vec{w})
\end{aligned} \tag{4.8}$$

Note that only those terms in the box change from line to line. If we think of Equation 4.8 as a "vector of equations," they state that each element should be zero (left-hand side) and equal to -2 times

the sum of "something that varies per line" times "something that is the same on each line" (right-hand side). Thus, instead of writing down an equation for each line separately, we can use vector-by-scalar multiplication to calculate the terms inside the sum simultaneously for all lines and then sum those together:

$$\vec{0} = -2\sum_{i=1}^{m} \begin{bmatrix} x_{i,0} \\ x_{i,1} \\ \vdots \\ x_{i,n} \end{bmatrix} (y_i - \vec{x}_i{}^\top \vec{w}) = -2\sum_{i=1}^{m} \vec{x}_i \, (y_i - \vec{x}_i{}^\top \vec{w}) \tag{4.9}$$

The right-hand side of this equation is the **gradient** of the loss function (with respect to $\vec{w}$): It is the vector we get from stacking together the right-hand sides of all of the lines in Equation 4.8, which are the partial derivatives of L with respect to each element in $\vec{w}$.

The gradient of a (scalar) function is a vector that points in the direction that the function increases locally the fastest. Its magnitude is equal to the slope in this direction. Equation 4.9 states that the gradient of L should be equal to the zero vector: The function does not increase (or decrease) in any direction locally.

Gradients obey mostly the same rules as derivatives (because they are vectors of derivatives), such as the chain rule. So, we could have derived Equation 4.9 directly with gradients:

Further information

Recall, the gradient of L with respect to $\vec{w}$ is written as $\nabla_{\vec{w}} L$ (or just ∇L when it is with respect to the argument of L) and is defined as

$$\nabla_{\vec{w}} L = \begin{bmatrix} \frac{\partial L}{\partial w_0} \\ \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_n} \end{bmatrix}.$$

$$\begin{aligned}
\vec{0} &= \nabla_{\vec{w}} L \\
&= \nabla_{\vec{w}} \left(\sum_{i=1}^{m} (y_i - \vec{x}_i{}^\top \vec{w})^2 \right) \\
&= \sum_{i=1}^{m} \nabla_{\vec{w}} \left((y_i - \vec{x}_i{}^\top \vec{w})^2 \right) \\
&= \sum_{i=1}^{m} \nabla_{\vec{w}} \, (y_i - \vec{x}_i{}^\top \vec{w}) \, (2 \, (y_i - \vec{x}_i{}^\top \vec{w})) && \text{chain rule, written in reverse order} \\
&= \sum_{i=1}^{m} (-\vec{x}_i) 2 \, (y_i - \vec{x}_i{}^\top \vec{w}) && \text{In general, } \nabla_{\vec{b}} \left(\vec{a}^\top \vec{b} \right) = \vec{a} \\
&= -2\sum_{i=1}^{m} \vec{x}_i \, (y_i - \vec{x}_i{}^\top \vec{w}) && (4.10)
\end{aligned}$$

Solving

We now can rearrange Equation 4.9 to solve for $\vec{w}$:

$$\begin{aligned}
\vec{0} &= -2\sum_{i=1}^{m} \vec{x}_i \left(y_i - \vec{x}_i{}^\top \vec{w}\right) \\
\vec{0} &= \sum_{i=1}^{m} \vec{x}_i \left(y_i - \vec{x}_i{}^\top \vec{w}\right) \\
&= \sum_{i=1}^{m} \left(\vec{x}_i y_i - \vec{x}_i \vec{x}_i{}^\top \vec{w}\right) \\
&= \sum_{i=1}^{m} \vec{x}_i y_i - \sum_{i=1}^{m} \vec{x}_i \vec{x}_i{}^\top \vec{w} \\
\sum_{i=1}^{m} \vec{x}_i \vec{x}_i{}^\top \vec{w} &= \sum_{i=1}^{m} \vec{x}_i y_i \\
\left(\sum_{i=1}^{m} \vec{x}_i \vec{x}_i{}^\top\right) \vec{w} &= \sum_{i=1}^{m} \vec{x}_i y_i \qquad \vec{w} \text{ does not depend on } i, \text{ so it can move outside the sum}
\end{aligned} \tag{4.11}$$

The right-hand side is $\sum_{i=1}^{m} \vec{x}_i y_i$ which is the sum of the feature vectors for each example multiplied by the corresponding target. This yields a vector. The left-hand side involves $\sum_{i=1}^{m} \vec{x}_i \vec{x}_i{}^\top$ which is the sum of the **outer product** of each feature vector with itself. Each outer product is a matrix containing all possible multiplications of two elements from $\vec{x}_i$. Because we will use these expressions again, we will define them as follows.

$$\begin{aligned}
\vec{c} &= \sum_{i=1}^{m} \vec{x}_i y_i = \boldsymbol{X}^\top \boldsymbol{Y} \\
\boldsymbol{A} &= \sum_{i=1}^{m} \vec{x}_i \vec{x}_i{}^\top = \boldsymbol{X}^\top \boldsymbol{X}
\end{aligned} \tag{4.12}$$

The right-most expressions use the sum implicit in matrix multiplication to perform the same operation. Refer to Equation 2.1 for the definitions of these data matrices. The outline of why these equalities are true is on the right. It is worth working through them yourself, as these types of transformations are common both in derivations and in code (to rearrange the computations to make faster on vector-oriented hardware). Note that the transposes are reversed between the vector version and matrix version because the matrix $\boldsymbol{X}$ consists of the vectors $\vec{x}_i{}^\top$, so it has a transpose "built in."

By separating the expressions that involve the data (Equation 4.12) from the weights, $\vec{w}$, the solution is immediate from Equation 4.11:

$$\begin{aligned}
\boldsymbol{A}\vec{w} &= \vec{c} \\
\vec{w} &= \boldsymbol{A}^{-1}\vec{c} \qquad \text{assuming } \boldsymbol{A} \text{ is nonsingular}
\end{aligned} \tag{4.13}$$

Further information

The outer product of a vector with itself has the following form:

$$\vec{a}\,\vec{a}^\top = \begin{bmatrix} a_1^2 & a_1a_2 & a_1a_3 & \cdots & a_1a_n \\ a_1a_2 & a_2^2 & a_2a_3 & \cdots & a_2a_n \\ a_1a_3 & a_2a_3 & a_3^2 & & a_2a_n \\ \vdots & & & \ddots & \vdots \\ a_1a_n & a_2a_n & a_3a_n & \cdots & a_n^2 \end{bmatrix}$$

Further information

Details for $\vec{c}$:

$$\begin{aligned}
&\boldsymbol{X}^\top \boldsymbol{Y} \\
&= \begin{bmatrix} | & | & & | \\ \vec{x}_1 & \vec{x}_2 & \cdots & \vec{x}_m \\ | & | & & | \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m x_{i,0}y_i \\ \sum_{i=1}^m x_{i,1}y_i \\ \vdots \\ \sum_{i=1}^m x_{i,n}y_i \end{bmatrix} \\
&= \sum_{i=1}^{m} \begin{bmatrix} x_{i,0}y_i \\ x_{i,1}y_i \\ \vdots \end{bmatrix} = \sum_{i=1}^{m} \vec{x}_i y_i = \vec{c}
\end{aligned}$$

Further information

Details for $\boldsymbol{A}$:

$$\begin{aligned}
&\boldsymbol{X}^\top \boldsymbol{X} \\
&= \begin{bmatrix} \sum_{i=1}^m x_{i,0}^2 & \sum_{i=1}^m x_{i,0}x_{i,1} & & \sum_{i=1}^m x_{i,0}x_{i,n} \\ \sum_{i=1}^m x_{i,0}x_{i,1} & \sum_{i=1}^m x_{i,1}^2 & \cdots & \sum_{i=1}^m x_{i,1}x_{i,n} \\ & \vdots & \ddots & \vdots \\ \sum_{i=1}^m x_{i,0}x_{i,n} & \sum_{i=1}^m x_{i,1}x_{i,n} & \cdots & \sum_{i=1}^m x_{i,n}^2 \end{bmatrix} \\
&= \sum_{i=1}^{m} \begin{bmatrix} x_{i,0}^2 & x_{i,0}x_{i,1} & & x_{i,0}x_{i,n} \\ x_{i,0}x_{i,1} & x_{i,1}^2 & \cdots & x_{i,1}x_{i,n} \\ \vdots & \ddots & & \vdots \\ x_{i,0}x_{i,n} & x_{i,1}x_{i,n} & \cdots & x_{i,n}^2 \end{bmatrix} \\
&= \sum_{i=1}^{m} \vec{x}_i \vec{x}_i{}^\top = \boldsymbol{A}
\end{aligned}$$

which can be expanded into

$$\vec{w} = \underbrace{(\boldsymbol{X}^\intercal \boldsymbol{X})^{-1} \boldsymbol{X}^\intercal}_{\text{pseudoinverse of } \boldsymbol{X}:\ \boldsymbol{X}^\dagger} \boldsymbol{Y} \tag{4.14}$$

The **pseudoinverse** of $\boldsymbol{X}$ is a generalization of the inverse to non-square matrices, also called the Moore-Penrose inverse. It is the most commonly used generalization of the inverse, primarily because it appears in least squares solutions as above. If $\boldsymbol{A}$ is singular, the expression above is not valid. However, the pseudoinverse is still defined and can be used in place of $(\boldsymbol{X}^\intercal \boldsymbol{X})^{-1} \boldsymbol{X}$, but it is not as straight-forward to define.

Thus, the learning algorithm is particularly simple in this case (Algorithm 4.2). To produce the weights, we compose the data into the matrices $\boldsymbol{X}$ and $\boldsymbol{Y}$ as in Equation 2.1. We then calculate the linear algebra expression in Equation 4.14.

We now have the weight vector $\vec{w}$ and so we can predict the target for any future example by building its feature vector $\vec{x}$ and evaluating $f(\vec{x}) = \vec{w}^\intercal \vec{x}$ (Algorithm 4.3).

LINEAR LEAST SQUARES LEARNING
In: $\boldsymbol{X}, \boldsymbol{Y}$
Out: $\vec{w}$

$\boldsymbol{A} \leftarrow \boldsymbol{X}^\intercal \boldsymbol{X}$
$\vec{c} \leftarrow \boldsymbol{X}^\intercal \boldsymbol{Y}$
$\vec{w} \leftarrow \boldsymbol{A}^{-1} \vec{c}$
return $\vec{w}$

Algorithm 4.2: Linear least squares learning algorithm. Linear algebra packages provide methods to compute the $\vec{w}$ that solves $\boldsymbol{A}\vec{w} = \vec{c}$ directly (instead of inverting $\boldsymbol{A}$ and multiplying by $\vec{c}$). This is faster and more accurate and should be used to calculate $\vec{w}$ in the third line.

LINEAR REGRESSION PREDICTION
In: $\vec{x}, \vec{w}$
Out: $y = f(\vec{x})$

return $\vec{x}^\intercal \vec{w}$

Algorithm 4.3: Linear regression prediction algorithm

Example

We will use the data from Table 2.1 to construct a rule to predict the land speed of a hippopotamus from its girth and ear wiggle rate. First, we collect the data into $\boldsymbol{X}$ and $\boldsymbol{Y}$, where we have added an initial (zeroth) column of ones to $\boldsymbol{X}$:

$$\boldsymbol{X} = \begin{bmatrix} 1.00 & 3.49 & 0.41 \\ 1.00 & 2.39 & 0.34 \\ 1.00 & 3.28 & 0.14 \\ 1.00 & 3.10 & 0.31 \\ 1.00 & 3.01 & 0.27 \\ 1.00 & 3.11 & 0.53 \\ 1.00 & 2.57 & 0.53 \\ 1.00 & 3.07 & 0.38 \\ 1.00 & 2.54 & 0.36 \\ 1.00 & 3.64 & 0.05 \end{bmatrix} \qquad \boldsymbol{Y} = \begin{bmatrix} 12.01 \\ 8.49 \\ 17.81 \\ 25.67 \\ 22.89 \\ 26.09 \\ 18.53 \\ 24.47 \\ 15.76 \\ 8.98 \end{bmatrix} \tag{4.15}$$

We then calculate $\vec{c}$ and $\boldsymbol{A}$ using the data matrices above and Equation 4.12:

$$\vec{c} = \begin{bmatrix} 180.70 \\ 545.70 \\ 63.51 \end{bmatrix} \qquad \boldsymbol{A} = \begin{bmatrix} 10.00 & 30.20 & 3.32 \\ 30.20 & 92.72 & 9.75 \\ 3.32 & 9.75 & 1.31 \end{bmatrix} \tag{4.16}$$

Further information
Because the values presented in this example are only shown to two digits of precision after the decimal point, but the calculuations are done with the full precision, you may end up with slightly different results if you perform the same calculations using only the precision shown.

Finally, we use Equation 4.13 to find the weight vector:

$$\vec{w} = \boldsymbol{A}^{-1}\vec{c} = \begin{bmatrix} 11.02 & -3.00 & -5.58 \\ -3.00 & 0.87 & 1.15 \\ -5.58 & 1.15 & 6.33 \end{bmatrix} \begin{bmatrix} 180.70 \\ 545.70 \\ 63.51 \end{bmatrix} = \begin{bmatrix} -1.53 \\ 4.04 \\ 22.26 \end{bmatrix} \tag{4.17}$$

Learning is done. Our learned function is

$$f(\vec{x}) = \vec{w}^{\mathsf{T}}\vec{x} = -1.53 + 4.04x_1 + 22.26x_2 \ . \tag{4.18}$$

Further information
Recall, that $\vec{x}$ has an additional 1 added to the beginning, to match the 1 added to all training examples.

If we later encounter a hippopotamus and (somehow) measure its girth to be 2.97 meters and its ear wiggle rate to be 0.48 Hertz, we estimate its running speed in kilometers per hour to be

$$f\left(\begin{bmatrix} 1 \\ 2.97 \\ 0.48 \end{bmatrix}\right) = -1.53 + 4.04 \cdot 2.97 + 22.26 \cdot 0.48 = 21.15 \ . \tag{4.19}$$

Exercises

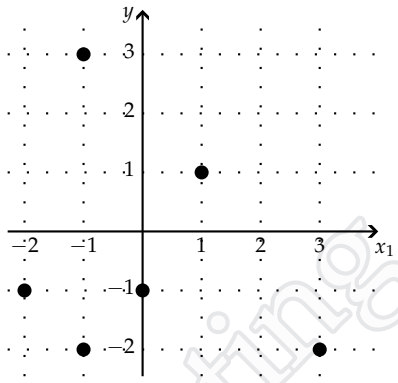

Figure 4.4: Dataset for Exercise 4.1

Exercise 4.1

Consider the six data points in Figure 4.4

a. What is the prediction rule, $f(x_1)$, that linear least squares would generate from this training data?
b. Plot this rule on the same axes as the data

Exercise 4.2

Consider the training set below.

$$\boldsymbol{X} = \begin{bmatrix} 2.1 & -0.5 & 1.1 \\ 5.5 & -1.2 & -1.2 \\ -1.8 & 0.0 & 0.1 \\ 2.2 & 1.6 & 1.3 \end{bmatrix} \qquad \boldsymbol{Y} = \begin{bmatrix} 3.0 \\ -1.1 \\ 2.7 \\ 4.1 \end{bmatrix}$$

a. What is the linear least-squares regression fit to these data? (Use a linear algebra software package or library to compute the answer.)
b. What is the training error?
c. Do you think this training error is indicative of the testing error? Why or why not?

Exercise 4.3

Prove for linear least-squares regression, if the average value of each feature in the training data is 0, w_0 (the intercept term) is the average y value in the training data.

It may help to know that if you have a block-diagonal matrix, its inverse is also block-diagonal:

$$\begin{bmatrix} \boldsymbol{B_1} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{B_2} \end{bmatrix}^{-1} = \begin{bmatrix} \boldsymbol{B_1}^{-1} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{B_2}^{-1} \end{bmatrix}$$

Exercise 4.4

Prove that the solution of Equation 4.14 is a minimum of the loss function (not a maximum or saddle point).

To do this, you should show that the Hessian (matrix of second derivatives) is positive semi-definite. The Hessian is positive semi-definite if

$$H_{j,k}(\vec{w}) = \frac{\partial^2 L(\vec{w})}{\partial w_j \partial w_k}$$

and

$$\vec{d}^{\top} \boldsymbol{H}(\vec{w})\, \vec{d} \geq 0 \qquad \text{for all } \vec{d}$$

which is the same as saying that the loss function's directional second derivative is non-negative in every direction around the solution to linear least-squares regression, $\vec{w}$.

Exercise 4.5

a. Figure 4.5 has a simple one-dimensional regression problem with the initial constant feature (x_0) already added. Compute the regression coefficients (weights) for this problem. You should not need a calculator or any complex matrix inversion.
b. Plot the loss function as a function of the weight vector, $\begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$. Use a range of -4 to 4 for both w_0 and w_1. This is a three-dimensional plot and most easily done with software.
c. What if we add an additional feature, x_2, that is always twice the value of x_1? Given that we do not need the intercept term, w_0, (see part a and Exercise 4.3) we will drop x_0 to keep a two-dimensional dataset, shown in Figure 4.6.
   Make the same plot as in part b, but for this new dataset.
d. What does the plot in part c imply about the linear least-squares solution for this dataset? How does that manefest itself in finding the weight vector algebraically?

x_0	x_1	y
1.00	−2.00	4.00
1.00	−1.00	3.00
1.00	−1.00	1.00
1.00	0.00	2.00
1.00	1.00	−5.00
1.00	3.00	−5.00

Figure 4.5: Toy regression dataset

x_1	x_2	y
−2.00	−4.00	4.00
−1.00	−2.00	3.00
−1.00	−2.00	1.00
0.00	0.00	2.00
1.00	2.00	−5.00
3.00	6.00	−5.00

Figure 4.6: Second toy regression dataset